Unidad 4
Actividad Práctica

En las distintas oportunidades que han presentado su aplicación, los equipos han indicado el tipo de optimización del ciclo de desarrollo de software que implementaron en su trabajo. Dado que tienen conocimiento de la temática correspondiente a la unidad 4, las actividades a realizar son:

- Leer el material.
- Indicar el tipo de optimización aplicado, justificar su elección y herramientas utilizadas.
- En el caso de no haber implementado optimización del ciclo de desarrollo, indicar como optimizaría el ciclo de desarrollo, justificar la respuesta.
- La respuesta presentada se deberá mostrar en la aplicación el día 11 de noviembre.

Entrega:
Parte 1: Responder en la tarea habilitado para cada comisión en Unidad 4. La actividad estará disponible hasta el día 6 de octubre a las 18.00 hs para la comisión de la tarde y hasta las 22.30 hs para la comisión de la noche. La entrega es en formato .pdf y debe ser individual.

Parte 2: Responder en la tarea habilitado para cada comisión en Unidad 4 . La actividad estará disponible hasta el día 6 de octubre a las 18.00 hs para la comisión de la tarde y hasta las 22.30 hs para la comisión de la noche. La entrega es en formato .pdf y debe ser por equipo.

PARTE-1

Actividad 1:
Indicar la verdad y falsedad de:

Contenido	V o F
Flujo típico de Integración Continua 1. **Desarrollador hace un cambio** en su código y lo sube (push) al repositorio. 2. El **servidor de CI detecta el cambio** y ejecuta automáticamente: o Compilación del código. o Ejecución de pruebas unitarias y de integración. o Análisis de calidad del código (linting, cobertura, vulnerabilidades). 3. Si todo pasa correctamente → el cambio puede **fusionarse (merge)** en la rama principal. **4.** Si hay fallos → el sistema **notifica** al equipo (correo, Slack, etc.).	**V**
Proceso de Integración continua: • Los desarrolladores trabajan en ramas de código separadas y las integran frecuentemente en la rama principal. • Un servidor de integración continua automáticamente compila el código y ejecuta pruebas unitarias y de integración para verificar los cambios. • Si hay un error, se notifica al desarrollador para que lo solucione rápidamente, evitando problemas mayores.	**V**
Principios clave de la CI • Integrar con frecuencia (varias veces al día, idealmente). • Mantener el código en un estado siempre **"listo para desplegar"**. • Automatizar las pruebas y validaciones. • Usar un repositorio compartido y control de versiones (Git). • Revisar los resultados del CI antes de aprobar merges.	**V**
Relación con Entrega Continua (CD)	**F**

• **Integración Continua (CI)**: asegura que el código nuevo funcione y se integre correctamente. • **Entrega Continua (CD)**: automatiza el despliegue del código en entornos de prueba o producción. En conjunto, forman el flujo **CI/CD**, clave en DevOps.	
Flujo típico de un pipeline CI/CD Commit → Build → Test → Package → Deploy → Notify **Ejemplo:** 1. Un desarrollador hace un git push. 2. Se ejecutan pruebas unitarias y de integración. 3. Si todo pasa, se genera un paquete o contenedor. 4. El sistema despliega la nueva versión en el entorno de prueba o producción. **5.** Se notifica al equipo del resultado.	**V**
Etapas típicas del despliegue automatizado 1. **Build (Construcción):** compilar o empaquetar el código fuente. 2. **Test (Pruebas):** verificar funcionalidad antes del despliegue. 3. **Package (Empaquetado):** generar artefactos (por ejemplo, .jar, .zip, o imágenes Docker). 4. **Deploy (Despliegue):** copiar o publicar el artefacto en el entorno. **5. Verify (Verificación):** comprobar que el sistema está activo y responde correctamente.	**V**

Fundamentación de las respuestas (según la teoría de la Unidad 4):

Flujo típico de Integración Continua – V: Coincide con el flujo Desarrollador → Commit → Repositorio → Build → Pruebas → Resultado. La compilación, las pruebas y el análisis de calidad los ejecuta el pipeline automáticamente; si todo pasa, el cambio avanza y, si hay un error, el equipo es notificado para corregirlo.

Proceso de Integración continua – V: CI consiste en integrar con frecuencia los cambios en un repositorio compartido y verificarlos con builds y pruebas automatizadas. Al integrar cambios pequeños los problemas se localizan más fácil y se evita que se acumulen.

Principios clave de la CI – V: Integrar varias veces al día, automatizar build y pruebas, usar un repositorio compartido con control de versiones, revisar los resultados del CI antes del merge y mantener el software en un estado siempre integrable y liberable.

Relación con Entrega Continua (CD) – F: La Entrega Continua mantiene el software preparado para liberarse, pero la liberación final a producción puede requerir aprobación manual. Lo que automatiza el despliegue a producción es el Despliegue Continuo, por lo que Delivery no implica necesariamente despliegue automático. El resto del enunciado (CI y CD forman el flujo CI/CD en DevOps) es correcto.

Flujo típico de un pipeline CI/CD – V: Responde a la cadena Build → Test → Package → Deploy: tras las pruebas se genera el paquete o la imagen, se despliega en el entorno de destino y el equipo recibe la notificación del resultado.

Etapas típicas del despliegue automatizado – V: Build, Test, Package y Deploy son etapas del pipeline; la verificación posterior corresponde a la ejecución de pruebas después del despliegue y al monitoreo.

Actividad 2: Completar el siguiente cuadro con la descripción que corresponda a cada beneficio de Integración continua

Beneficio	Descripción
Detección temprana de errores	Los fallos se detectan en minutos, no semanas después.
Reducción de conflictos de integración	Al integrar frecuentemente, se evitan "infiernos de merge".
Mayor calidad del software	Las pruebas automáticas garantizan estabilidad.
Entrega más rápida	El producto está siempre en condiciones de ser desplegado.
Transparencia	Todos pueden ver el estado del proyecto (builds, pruebas, cobertura).

Descripciones:

Los fallos se detectan en minutos, no semanas después.
Al integrar frecuentemente, se evitan “infiernos de merge”.
Las pruebas automáticas garantizan estabilidad.
El producto está siempre en condiciones de ser desplegado.
Todos pueden ver el estado del proyecto (builds, pruebas, cobertura).

Actividad 3:

A parte de las herramientas CI/DI que se mencionan en el texto, proponer una herramienta que a criterio del equipo se debería aplicar. Justificar la respuesta.

Herramienta propuesta: SonarQube (como complemento de GitHub Actions, GitLab CI/CD o Jenkins, que son las plataformas que ejecutan el pipeline).

Justificación:

1) Está ubicada entre las herramientas de la etapa de prueba, destinadas a analizar la base de código, por lo que completa a los ejecutores de pipelines con un control específico de calidad.

2) Permite implementar la política de **quality gate**: el pipeline solo avanza a la siguiente etapa si se cumplen determinados criterios de calidad, combinando las pruebas automatizadas con el análisis de calidad del código.

3) Responde a las buenas prácticas de la unidad: incorporar análisis de seguridad, revisar periódicamente el desempeño del pipeline y sostener el código limpio. Además, el análisis estático y la verificación de estilo pueden ejecutarse automáticamente al crear un Pull Request, validando el código antes de incorporarlo a la rama principal.

4) Beneficio esperado: mayor calidad del software y detección temprana de errores, con menos carga de revisión manual y retroalimentación inmediata para el equipo.

Ubicación en el pipeline: Source → Build → Test (pruebas unitarias, de integración y análisis de calidad con SonarQube) → Package → Security → Deploy. Se ejecuta ante un push o al crear un Pull Request y, si no se cumple el **quality gate**, el pipeline se detiene y se notifica al equipo.

Actividad 4

Responder los siguientes interrogantes:

- Revea la unidad 3- Buenas prácticas de programación y establezca de ser posible las relaciones con la unidad 4.

  Las buenas prácticas de programación (código limpio, refactorización, revisión de código, convenciones y pruebas) se relacionan con la Unidad 4 de la siguiente manera:

Código limpio y refactorización: la unidad propone refactorizar con frecuencia para mejorar la legibilidad, la estructura interna y la mantenibilidad sin alterar la funcionalidad externa, y ubica el código limpio junto con la Integración Continua en la fase de Desarrollo. La CI aporta las pruebas automáticas que permiten verificar, ante cada cambio, que el código refactorizado sigue funcionando.

Revisión de código: se integra al flujo mediante Pull Requests / Merge Requests (Pull Request → CI → Code Review → Merge), con análisis estático y verificación de estilo automáticos y con políticas de protección de ramas.

Commits pequeños y comprensibles: es una de las buenas prácticas de CI/CD y se complementa con la integración frecuente: al ser reducido el conjunto de cambios, los problemas son más fáciles de localizar.

Pruebas automatizadas: las pruebas unitarias verifican pequeñas unidades de código (funciones o métodos) y, junto con las de integración, funcionales y de interfaz, forman parte del pipeline.

Convenciones y documentación: las Wikis permiten documentar las convenciones de programación, la arquitectura, las decisiones técnicas y las guías para nuevos integrantes; documentar el proceso es también una buena práctica.

Arquitectura modular: una arquitectura sólida y modular facilita el desarrollo paralelo, el mantenimiento y las futuras ampliaciones, lo que acompaña el trabajo en ramas separadas y la integración frecuente.

Configuración separada del código y manejo de datos sensibles: las variables de entorno permiten separar la configuración del código fuente y los secrets no deben almacenarse en el repositorio.

- ¿De qué manera la implementación de CI/CD modifica el proceso tradicional de desarrollo de software y qué beneficios aporta en términos de calidad, tiempo, trazabilidad y productividad?

  Cómo cambia el proceso. En el modelo tradicional los ciclos de implementación son largos, las pruebas se realizan principalmente al final del proyecto y varios desarrolladores pueden trabajar semanas o meses en ramas diferentes para integrar luego todos los cambios, lo que produce conflictos difíciles de resolver. Además, los despliegues manuales son riesgosos, llevan mucho tiempo y requieren experiencia y documentación. Con CI/CD, en cambio, los cambios se integran con frecuencia en un repositorio compartido y cada uno se integra, compila, prueba y valida automáticamente. El trabajo pasa a organizarse como un pipeline (Source → Build → Test → Package → Security → Deploy) que se activa ante eventos como un push o un Pull Request, con artefactos reutilizables entre etapas, estrategias de despliegue controladas y métricas para evaluar el proceso.

  Calidad: las pruebas automatizadas y las validaciones continuas mantienen un nivel de calidad más consistente; los quality gates impiden avanzar si no se cumplen los criterios; y el Pull Request permite ejecutar análisis estático, de estilo y de seguridad antes del merge.

  Tiempo: los errores se detectan rápidamente, el desarrollador recibe retroalimentación rápida y se reducen las tareas manuales. En el ejemplo de la sección 14, de 52 horas de ciclo sólo 12 corresponden a desarrollo y corrección; las 40 restantes son esperas (revisión: 20 h; pruebas: 12 h) y despliegue manual (8 h), que la automatización puede reducir. Desplegar pequeños lotes de cambios con frecuencia disminuye además el riesgo de los lanzamientos.

Trazabilidad: el repositorio permite relacionar commit, autor, rama, Issue, Pull/Merge Request, pipeline, pruebas, artefacto y despliegue. La gestión de artefactos permite identificar exactamente qué versión fue desplegada y el flujo Issue → Rama → Commit → Pull Request → Pipeline → Merge → Deploy → Issue cerrada brinda trazabilidad completa del trabajo.

Productividad: se automatizan las tareas repetitivas, cualquier integrante del equipo puede realizar un despliegue con sólo pulsar un botón sin conocimientos técnicos especiales y se libera tiempo para tareas más importantes. Además, métricas como Lead Time y Deployment Frequency permiten medir la velocidad de entrega, detectar cuellos de botella y orientar la mejora continua.

- ¿Qué mejoras introduce CI/CD en el trabajo colaborativo y la calidad del software?

  Trabajo colaborativo:

  Repositorio compartido: todo el equipo integra frecuentemente sus cambios en un mismo lugar, lo que facilita el trabajo colaborativo.

  Pull Requests / Merge Requests: los cambios son analizados por el equipo antes de incorporarse a la rama principal.

  Issues, Wikis y tableros Kanban: se registran y asignan las tareas, se documenta el conocimiento compartido y se visualiza el flujo de trabajo (trabajo pendiente, en ejecución, bloqueado, en revisión y terminado), con límite de Work In Progress (WIP) para evitar la sobrecarga.

  Cultura DevOps: la automatización reduce las barreras entre Desarrollo, Operaciones, Testing/QA, Seguridad, Infraestructura y Gestión de proyectos, y genera un flujo de trabajo más integrado. La mejora combina Personas + Procesos + Tecnología + Automatización + Métricas. Las metodologías ágiles promueven además la comunicación abierta y fluida entre desarrolladores, testers y operaciones.

  Calidad del software:

  Detección temprana de errores y retroalimentación rápida, que evitan que los problemas se acumulen y sean más costosos de corregir.

  Automatización de build y pruebas (unitarias, de integración, funcionales y de interfaz) con pipelines reproducibles.

  Revisión de código, análisis de seguridad y protección de ramas.

  Monitoreo y observabilidad posteriores al despliegue, para detectar y solucionar rápidamente problemas de rendimiento o bugs en producción, y estrategias de rollback ante versiones con problemas.

- ¿Cómo se podría incorporar la IA a los contenidos desarrollados en la Unidad 4?

  La Unidad 4 no desarrolla contenidos específicos de Inteligencia Artificial; por ello, se propone incorporarla como **asistente** en las etapas que la unidad ya describe, manteniendo siempre el control humano y sometiendo lo que genere la IA a las mismas validaciones del pipeline (pruebas, quality gates y code review):

  Pull Requests y code review: pre-revisión del cambio para sugerir mejoras de estilo y posibles malas prácticas antes de la revisión del equipo, que mantiene la decisión final del merge.

  Automatización de pruebas: ayuda para proponer casos de prueba unitarios y de integración y señalar partes del código con poca cobertura.

Pipelines y builds: análisis de los registros de ejecuciones fallidas para explicar la causa probable del error y sugerir su corrección, acelerando la retroalimentación al equipo.

Métricas de productividad: análisis de los datos de Lead Time y Deployment Frequency para detectar cuellos de botella, como las esperas de revisión o de pruebas del ejemplo de la sección 14, y anticipar demoras.

Issues y tablero Kanban: redacción, clasificación y priorización de Issues (por ejemplo, con la técnica MoSCoW) y resumen del estado del tablero.

Wikis y documentación: generación y actualización de la documentación (instalación, configuración, arquitectura, guías para nuevos integrantes) a partir de los cambios realizados.

Seguridad y variables de entorno: detección de credenciales o tokens subidos por error al repositorio, en línea con la práctica de no almacenar secretos en él.

Monitoreo y despliegue: detección de comportamientos anormales luego de un despliegue (por ejemplo, en una estrategia Canary) para apoyar la decisión de ejecutar un rollback.